A guarded core for intuitionistic primitive recursive arithmetic

Removing the introduction choice while preserving both interpretations

Research prepared for Vladimir Reshetnikov
with ChatGPT

10 November 2026

Abstract

The current ConciseIPRA presentation permits both case and progress introduction at literal equations. This report removes that choice from the canonical calculus. An atomic matrix immediately below the paired binder must have the form $I(y, 0; u.h) = I(y, 0; u.k)$, whose zero instance is a computation theorem. The one introduction schema then uses progress at an equation and cases at a compound matrix. Padding reconstructs the missing logical uses; a short argument with implication between equations recovers unrestricted contextual equation induction without assuming predecessor or pairing. Explicit proof translations and both native composites transfer the existing whole-domain bi-interpretation with full first-order iPRA.

Under the repository's canonical-statement checking convention, the direct checker decreases from 529 to 528 normalized Python bytes. An additional implementation composition gives 527 bytes. Separate profiles give 260 significant tokens and 286 AST nodes, compared with the current reported minima 261 and 289. These minima belong to different programs. Gzip-9 sizes are recorded as supplementary measurements.

The gain is small and no global minimality theorem is claimed. The canonical grammar and the broader administrative certificate language must be distinguished: an unrestricted atomic binder has a different sound interpretation, and the entire unrestricted syntax is not asserted to be bi-interpretable with iPRA. A separately measured standalone checker with explicit canonical-goal validation is also included. The new metatheorems are written proofs, supported by executable certificates; they have not been formalized in a proof assistant.

Contents

1 Result and comparison contract **3**

2 A guarded core with a deterministic introduction rule **4**

3 The ordinary theory and the iPRA bridge **10**

4 Executable checker and its exact theorem **11**

5 Novelty, limits, and reproduction **15**

Attribution and inherited proofs **16**

A Complementary introductions with one local assumption **16**

B Arithmetic strength and both ordinary interpretation composites **21**

C Internal scalar predecessor bootstrap **23**

D From literal-equation induction to full intuitionistic PRA **27**

E Pairing, primitive recursion, and internal numerical composites **32**

1 Result and comparison contract

The comparison is frozen at ProveIt commit

`b17dd5487f34adea473a0204c5320fb3eeb8910e`.

The current proposal is the directory `Logic/PrimitiveRecursiveArithmetic/ConciseIPRA`. Its README describes the native scalar core and its smallest complete checker profiles. We inspected that README, the current Lean source, the principal reports and claim files, and the exact comparison programs. The three comparison programs are supplied unchanged in `reference/`. This is a comparison with the pinned proposal and its documented frontier; it is not a new exhaustive search through every conceivable checker.[1, 2, 3, 4]

The mathematical result concerns the *canonical guarded calculus* $\mathcal{G}$. Its terms, number domain and equality remain those of the scalar core. The new ingredients are a restriction on atomic paired matrices, a deterministic introduction rule, and explicit translations showing that the restriction loses no expressive or proof-theoretic strength. The arithmetic compilation into iPRA is inherited, not newly attributed to this report.

The repository's arity-encoded checkers accept a larger class of raw arrays than the canonical object language. Their completeness theorem is stated for canonical statements; surplus shapes have specified administrative readings. The present compact comparison uses the same kind of contract. In particular, the compact program does not reject every noncanonical public goal. This qualification is needed for the mathematical statement, not merely for malformed-input handling. Section 4 specifies it precisely and supplies a stricter alternative with its full additional cost.

1.1 Measured programs

All figures below charge the entire standalone Python module: traversal, substitution, scopes, context handling, the arithmetic equations, logical rules, JSON input and the acceptance driver. The common Python runtime and standard JSON parser are excluded, as in the current proposal. Source is normalized by removing docstrings and applying `ast.unparse`, with one final newline. The environment is CPython 3.12.14. Gzip uses maximum compression, a zero timestamp, and includes its wrapper.

Standalone program	Bytes	Tokens	AST	Gzip 9
Current byte profile	529	266	310	333
Current token profile	576	261	292	345
Current AST profile	613	263	289	349
Direct guarded checker	528	265	308	332
Guarded byte profile	527	266	307	335
Guarded token profile	575	260	290	345
Guarded AST profile	607	263	286	349

The direct guarded checker in `code/checker.py` makes the mathematical change visible in isolation: one significant token and two AST nodes are removed from the 529-byte reference. The 527-byte profile also composes the major-premise check with the hypothesis supply expression. This second change preserves successful executions and is an implementation simplification, not a further axiom reduction.

The coordinate envelope becomes

$$(\text{normalized bytes}, \text{significant tokens}, \text{AST nodes}) = (527, 260, 286).$$

It is achieved by three programs, not by a single checker. The respective improvements over the documented 529/261/289 envelope are two bytes, one token and three AST nodes. These are observed upper bounds on an unknown optimum. They do not establish a lower bound, unique optimum, or absolute simplest axiomatization.

The direct lexical checker is the easiest compact profile to inspect. The generic-loop token and AST profiles retain the inherited broad exception-based traversal, including its qualification that some malformed strings can diverge in an ideal unlimited-stack model. On ordinary CPython they fail by an exception or resource limit. Acceptance always means normal exit with exactly `ACCEPT`; assertions are not used as logical gates.

2 A guarded core with a deterministic introduction rule

The change is to the public formula language and the introduction rule, not to the numerical domain. An atomic matrix immediately below a paired binder must have a fixed, syntactically recognizable zero guard. Such a matrix has a provable zero instance. Consequently it needs only the progress introduction; the separate atomic case introduction can be deleted. Compound matrices retain the case introduction.

2.1 Complete syntax, axioms, and inference rules

Numerical terms are those of the inherited scalar calculus:

$$t ::= x \mid 0 \mid S(t) \mid I(n, a; u.h).$$

Only the displayed variable u is bound in h. There is one numerical sort, fixed finite term syntax, and capture-free substitution. Put

$$C(v, a, b) := I(v, a; u.b), \qquad 1 := S0,$$

where u is fresh. For a literal equation $E \equiv (s = t)$, define

$$G_v E \;:=\; \big(C(v, 0, s) = C(v, 0, t)\big). \tag{Guard}$$

The displayed abbreviations expand into terms of the given syntax.

Call a literal equation a *zero-initialized matrix in y* when it has the syntactic form

$$B(x, y) \equiv I(y, 0; u.h) = I(y, 0; u.k). \tag{Canonical atomic matrix}$$

Here h, k may depend arbitrarily on the state variable u, on x, y, and on fixed parameters; state binders are renamed as necessary. The selector guard $G_y E$ is a special case, with state-independent updates. Every matrix of (Canonical atomic matrix) has the theorem $B(x, 0)$ by the two zero computations and equality.

The canonical formulas of $\mathcal{G}$ are generated as follows:

1. Every literal equation $s = t$ is a formula.

2. $\mathsf{D}_{x,y}B$ is a formula when B is already a compound canonical formula (its root is D).
3. $\mathsf{D}_{x,y}B(x,y)$ is a formula whenever B has the zero-initialized shape (Canonical atomic matrix).

The bound variables x, y are distinct. Rule 3 is the restriction on atomic matrices. It does not require an equality decision procedure or a proof that an arbitrary base is true: the guard has the displayed literal shape. The designated contradiction is $0 = 1$.

Retained contexts contain reusable clauses $\langle A; y.B\rangle$, where A, B are canonical formulas and $y \notin \mathrm{FV}(A)$. An ordinary assumption F abbreviates the constant clause $\langle H; v.F\rangle$, where v is fresh and H is the fixed computation theorem $I(0,0;u.0) = 0$.

The two nonlogical axiom schemas, available in every context, are

$$I(0,a;u.h) = a, \qquad I(Sn,a;u.h) = h[I(n,a;u.h)/u]. \tag{I0,IS}$$

Besides these, there are precisely the following rules:

$$\frac{\Gamma \vdash A}{\Gamma \vdash B(t)} \quad \text{if } \langle A; y.B(y)\rangle \in \Gamma \qquad \text{(Use)},$$

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)} \qquad \text{(Tr)},$$

$$\frac{\Gamma \vdash \mathsf{D}_{x,y}B(x,y) \qquad \Gamma, \langle B(a,0); y.B(a,y)\rangle \vdash F}{\Gamma \vdash F} \qquad \text{(DE)},$$

and the single, root-determined introduction schema

$$\frac{\Gamma, B(t,\eta_B(z)) \vdash B(t,Sz)}{\Gamma \vdash \mathsf{D}_{x,y}B(x,y)}, \qquad \eta_B(z) = \begin{cases} z, & B \text{ is a literal equation,} \\ 0, & B \text{ is compound.} \end{cases} \tag{DI$_{\mathrm{root}}$}$$

There is no certificate choice between these two cases. At an atomic matrix the guard condition on the conclusion is mandatory in the canonical calculus. In DE, a is fresh for Γ, F and the fixed parameters. In DI, z is fresh for Γ, t and the fixed parameters. The displayed local resources are discharged. Transport ranges over canonical formula templates, and capture-free numerical substitution preserves their guard shape.

There are no additional primitive logical rules, reflexivity axioms, successor axioms, coverage axioms, or induction schemas. Weakening, exchange, contraction, renaming, numerical substitution, and ordinary-assumption cut are admissible by induction on derivations. For cut, replace a use of the corresponding constant clause by the supplied proof, renaming eigenparameters first.

2.2 Padding, equation implication, and numerical cases

Write $A \dashv\vdash B$ for interderivability under arbitrary additional retained context. The following preliminary arguments use no contextual induction rule.

Lemma 2.1 (Equality and guard endpoints)**.** *Reflexivity, symmetry, transitivity, numerical congruence, and their usual contextual uses are derivable. Moreover,*

$$\vdash G_0E, \qquad G_{Sz}E \dashv\vdash E.$$

Proof. For reflexivity, put $r = I(0,t;u.0)$. The computation equation $r = t$ serves as both premises of transport with motive $v = t$, yielding $t = t$. Standard equality motives then give the remaining equality rules. The two guard claims follow from the two computation equations for C, together with equality transport. □

For an equation E let

$$L(E) := \mathsf{D}_{u,v} G_v E, \qquad P(A) := \begin{cases} L(A), & A \text{ is an equation,} \\ A, & A \text{ is compound.} \end{cases} \tag{Padding}$$

The newly bound u, v are fresh for E. In particular, $P(A)$ is always compound.

Lemma 2.2 (Padding)**.** *$L(E) \dashv\vdash E$, and therefore $P(A) \dashv\vdash A$.*

Proof. Under E, atomic progress introduction derives $L(E)$: its target $G_{Sz}E$ follows from the retained E, independently of the local induction assumption. Conversely, DE on $L(E)$ opens $\langle G_0 E; v.G_v E\rangle$. Supply the guard-zero theorem and use the clause at 1; guard removal gives E. □

We will need to reason by numerical cases under an assumption that contains the case variable. It is essential not to violate the eigenvariable condition by directly retaining that assumption during case introduction. A small implication macro resolves this issue.

For equations $U \equiv (s = t)$ and $V \equiv (r = q)$ put

$$M_w(U,V) := (C(w,s,r) = C(w,t,q)), \qquad U \Rightarrow_g V := \mathsf{D}_{a,w} L(M_w(U,V)), \tag{Equation implication}$$

with fresh padding variables and unused a. The matrix of the outer binder is compound.

Lemma 2.3 (Implication between equations)**.** *The macros in* (Equation implication) *admit both rules*

$$\frac{\Gamma, U \vdash V}{\Gamma \vdash U \Rightarrow_g V}, \qquad \frac{\Gamma \vdash U \Rightarrow_g V \qquad \Gamma \vdash U}{\Gamma \vdash V}.$$

Proof. The endpoint identities for C and padding give $L(M_0(U,V)) \dashv\vdash U$ and $L(M_{Sz}(U,V)) \dashv\vdash V$. Compound case DI therefore gives introduction. DE, the first endpoint, Use at 1, and the second endpoint give elimination. These endpoint conversions use numerical transport with parameters outside the fresh padding binders; they do not assume a binder-congruence theorem. □

Lemma 2.4 (Numerical cases)**.** *For every canonical $A(y)$ the following rule is admissible:*

$$\frac{\Gamma \vdash A(0) \qquad \Gamma \vdash A(Sz)}{\Gamma \vdash A(t)}, \qquad z \text{ fresh for } \Gamma \text{ and the fixed parameters.} \tag{NC}$$

Proof. First suppose A is compound. Case DI with an unused witness coordinate introduces $\mathsf{D}_{u,y}A(y)$, ignoring its local base assumption in the successor premise. DE opens $\langle A(0); y.A(y)\rangle$; supply the given base proof and use it at t.

If A is an equation, apply this already proved compound case rule to $L(A(y))$. Padding converts the two premises to its endpoints and converts the conclusion back. No progress induction is being used in this construction beyond the special padding introductions. □

2.3 Recovery of contextual equation induction

Theorem 2.5 (Equation induction)**.** *For every literal equation $E(y)$, $\mathcal{G}$ derives*

$$\frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)}, \qquad z \text{ fresh for } \Gamma \text{ and the fixed parameters.} \tag{EI}$$

Proof. Put $J(y) = G_y E(y)$. We first prove the progress premise $\Gamma, J(z) \vdash J(Sz)$ without making an invalid contextual use of NC. Consider the compound formula

$$\Phi(y) := J(y) \Rightarrow_g E(Sy).$$

Under Γ, prove $\Phi(0)$ by equation-implication introduction: ignore its local hypothesis, use the supplied proof of $E(0)$, and apply the supplied step proof specialized at 0 to obtain $E(1)$.

For fresh w, prove $\Phi(Sw)$ by equation-implication introduction. Its local hypothesis $J(Sw)$ gives $E(Sw)$ by guard removal, and the supplied step proof specialized at Sw gives $E(SSw)$. These two branch proofs have retained context Γ, independent of the case variable. Thus NC legitimately gives $\Phi(z)$ under Γ. Under the local hypothesis $J(z)$, equation-implication elimination gives $E(Sz)$, hence $J(Sz)$ by the successor guard endpoint.

Atomic progress DI now introduces $\mathsf{D}_{u,y}J(y)$ with unused u. DE and the theorem $J(0)$ yield $J(Sw)$, and hence $E(Sw)$, for a fresh w. Finally NC for E uses the supplied base $E(0)$ and these successor instances to conclude $E(t)$.

The only induction-like premise used here is the single displayed guarded progress introduction. In particular, no predecessor, coverage disjunction, formula induction, or pre-existing EI is assumed. □

2.4 Exact translations to and from the inherited calculus

Let $\mathcal{C}$ be the inherited Choice-DI calculus: the same scalar terms, arbitrary formulas $s = t$ and $\mathsf{D}_{x,y}B$, the same computation, Use, DE, and transport rules, with case DI at every matrix and progress DI at literal equations. Define $T : \mathcal{C} \to \mathcal{G}$ on finite formulas by

$$T(s = t) = (s = t), \qquad T(\mathsf{D}_{x,y}B) = \mathsf{D}_{x,y}P(TB). \tag{Translation}$$

Each padding binder uses fresh names. This is a computable structural translation, preserves free numerical variables, commutes with capture-free numerical substitution up to bound-variable renaming, and translates every paired matrix to a compound matrix. Translate clauses by $T\langle A; y.B\rangle = \langle TA; y.TB\rangle$. Let $J : \mathcal{G} \to \mathcal{C}$ be literal inclusion.

Theorem 2.6 (Translation of derivations)**.**

$$\Gamma \vdash_{\mathcal{C}} A \Longrightarrow T\Gamma \vdash_{\mathcal{G}} TA, \qquad \Gamma \vdash_{\mathcal{G}} A \Longrightarrow J\Gamma \vdash_{\mathcal{C}} JA.$$

The transformations are effective on finite derivations.

Proof. Inclusion is immediate: every guarded rule is an allowed Choice-DI rule. For T, induct on a Choice-DI derivation.

Computation is unchanged. Use commutes with substitution, and transport uses the translated motive. A case DI premise translates to a proof from $TB(t, 0)$ to $TB(t, Sz)$. Under the local assumption $P(TB)(t, 0)$, unpad, apply that translated proof, and pad the result. Compound case DI then proves the required $\mathsf{D}_{x,y}P(TB)$.

For a progress DI with equation matrix E, its translated step is still $\Gamma, E(t, z) \vdash E(t, Sz)$. To introduce $\mathsf{D}_{x,y}L(E(x, y))$, use compound case DI. Its retained base $L(E(t, 0))$ supplies $E(t, 0)$. The recovered contextual EI, under that base, gives $E(t, Sz)$, which padding converts to the target $L(E(t, Sz))$.

For DE, the translated major is $\mathsf{D}_{x,y}P(TB)$. It opens the clause $\langle P(TB)(a, 0); y.P(TB)(a, y)\rangle$. The recursively translated minor expects $\langle TB(a, 0); y.TB(a, y)\rangle$. Replace every use of this expected clause by: pad its antecedent, use the opened clause at the same term, and unpad its consequent. This effective local proof replacement preserves the fresh witness condition and the conclusion. □

Lemma 2.7 (Compound-matrix congruence)**.** *If canonical compound matrices B, C are uniformly interderivable, then $\mathsf{D}_{x,y}B \dashv\vdash \mathsf{D}_{x,y}C$ in $\mathcal{G}$.*

Proof. Open the source binder at a fresh witness a. Introduce the target by compound case DI at that witness. Convert its retained base to the source base, use the opened source clause at Sz, and convert the successor instance to the target instance. Close DE. The reverse conversion proves the converse. □

Lemma 2.8 (The guarded atomic composite)**.** *For an admissible zero-initialized atomic matrix $B(x, y)$,*

$$\mathsf{D}_{x,y}B \dashv\vdash \mathsf{D}_{x,y}L(B) \quad \textit{in } \mathcal{G}. \tag{Atomic composite}$$

Proof. Forwards, open the source clause at a. Introduce the target by compound case DI: unpad its base, use the source clause at Sz, and pad the result.

Backwards, open the clause of $\mathsf{D}_{x,y}L(B)$ at a. The two zero computations prove $B(a, 0)$, and padding proves $L(B(a, 0))$. Thus the opened clause gives $L(B(a, Sz))$, and unpadding gives $B(a, Sz)$, independently of the local assumption $B(a, z)$. Atomic progress DI therefore introduces $\mathsf{D}_{x,y}B$ at witness a. Close DE. This argument uses the true base explicitly; it would fail for an arbitrary unguarded atomic matrix. □

Theorem 2.9 (Both composites and bi-interpretability)**.** *For all $\mathcal{C}$-formulas A and all canonical $\mathcal{G}$-formulas B,*

$$A \dashv\vdash_{\mathcal{C}} JT(A), \qquad B \dashv\vdash_{\mathcal{G}} TJ(B).$$

Consequently $\mathcal{C}$ and $\mathcal{G}$ admit effective structural intertranslations preserving their full consequence relations, with both native formula composites and literal identity numerical translations.

Proof. In $\mathcal{C}$, padding is interderivable with its argument, and case introduction proves binder congruence for arbitrary matrices. Structural induction gives the first displayed equivalence.

For the second, induct on a canonical B. Equations are unchanged. At a compound matrix, the induction hypothesis and compound-matrix congruence apply. At a guarded atomic matrix, the statement is exactly (Atomic composite). The equivalences are uniform under retained assumptions, so they extend to clauses and sequent contexts by the same local replacement of Use instances.

Both interpretations leave the numerical sort, every term, and numerical equality untouched. Both numerical composites therefore have the identity graph $x = y$; no quotient, retract, numeral restriction, or extra coding sort is involved. □

For the first-order bi-interpretability statement, use the ordinary realization that reads every canonical $\mathsf{D}_{x,y}B$ uniformly as $\exists x(B(x, 0) \to \forall y B(x, y))$ and leaves numerical equality and all scalar terms unchanged. Composing the displayed translations and composites with the inherited full-domain bridge between $\mathcal{C}$ and iPRA yields a bi-interpretation of this ordinary realization of $\mathcal{G}$ with full first-order iPRA. The contradiction $0 = 1$ is unchanged, so equiconsistency follows as well. The inherited arithmetic bridge supplies the primitive-recursive compiler and the ordinary logical interpretation; this section does not claim those inherited constructions as new.

2.5 Why an unrestricted checker can still be sound

There is a useful, but strictly separate, administrative calculus $\mathcal{U}$. It permits arbitrary atomic paired matrices and uses the same root-determined DI, with ordinary Q-style clauses on DE. It is not claimed that all of $\mathcal{U}$ is bi-interpretable with iPRA.

Give arbitrary administrative formulas their following ordinary interpretations:

$$(s = t)^\dagger = (s = t),$$

$$(\mathsf{D}_{x,y}B)^\dagger = \begin{cases} \exists x\, \forall y\, (B(x,y) \to B(x,Sy)), & B \text{ is a literal equation,} \\ \exists x\, (B^\dagger(x,0) \to \forall y\, B^\dagger(x,y)), & B \text{ is compound.} \end{cases} \quad \text{(Mixed administrative reading)}$$

Retained clauses keep the ordinary reading $A^\dagger \to \forall y\, B^\dagger$. If the raw codec also permits administrative Q-closures as formula-shaped objects, including under transport or a paired binder, extend this interpretation by $(Q_y B)^\dagger = B^\dagger(0) \to \forall y B^\dagger(y)$. Such a matrix is compound for root-determined DI. This adds no canonical object-formula constructor and preserves the following soundness argument.

Every $\mathcal{U}$ inference is sound in iPRA under this reading. Atomic DI directly proves its progressive reading; compound case DI uses numerical coverage. In atomic DE, contextual equation induction turns $\forall y(E(y) \to E(Sy))$ into $E(0) \to \forall y E(y)$, exactly the clause that DE supplies. All other rules are immediate, including transport: numerical substitution does not change whether a matrix is an equation.

On canonical zero-initialized matrices, the mixed and ordinary readings agree provably in iPRA. Indeed $B(x,0)$ is a theorem, so, with x fixed,

$$\forall y\, (B(x,y) \to B(x,Sy)) \quad \longleftrightarrow \quad B(x,0) \to \forall y\, B(x,y).$$

The forward direction is equation induction; the reverse uses the true base to obtain every instance. Structural induction extends this agreement to all canonical guarded formulas.

Thus administrative proofs with unguarded intermediate formulas are conservative on canonical guarded conclusions: translate an accepted derivation into iPRA, use the canonical agreement, and translate back through the established bi-interpretation. Conversely, every canonical guarded derivation is already an administrative derivation.

This justifies a checker contract consisting of soundness and completeness for canonical guarded statements while allowing broader administrative certificates. It does *not* make the raw checker a validator of the guarded public grammar. It also does not assert completeness for every noncanonical encoding under the mixed reading. If a comparison requires rejection of all noncanonical public goals, the corresponding syntax validator must be implemented and counted.

2.6 Why merely deleting the choice would not suffice

Without the guarded canonical language, root-deterministic DI does not preserve the original native consequence relation. Let

$$p(y) = I(y, 0; u.C(u, 1, 0)), \qquad E(y) \equiv (p(y) = 1),$$

and leave the existential witness coordinate unused. Computation gives $p(0) = 0, p(1) = 1, p(2) = 0$. The original case introduction proves $\mathsf{D}_{x,y}E(y)$: its base gives $0 = 1$, from which equality transport and selector computation prove every equation.

By contrast, the progressive reading of this formula is false in the standard natural numbers, because $E(1)$ holds and $E(2)$ does not. The mixed interpretation proves that it is not a theorem of the unrestricted root-deterministic calculus. The guard, the explicit translations, and both composite proofs are therefore substantive parts of the reduction, rather than optional notation.

3 The ordinary theory and the iPRA bridge

Here iPRA means full first-order intuitionistic arithmetic with a symbol for each fixed finite primitive-recursive description, the defining equations, the usual successor separation and injectivity axioms, and contextual quantifier-free induction. Function descriptions are external finite syntax; there is no object-language universal evaluator.

The ordinary scalar realization is the theory T_{eq} with intuitionistic first-order equality logic, the two iterator equations,

$$\forall x\,(x = 0 \vee \exists y\, x = Sy), \qquad (0 = S0) \to \bot,$$

and contextual induction for literal equations. Its numerical domain is the whole original domain, and its interpreted equality is actual equality. Every canonical guarded binder has the uniform ordinary reading

$$\big(\mathsf{D}_{x,y}B\big)^{\circ} = \exists x\big(B^{\circ}(x,0) \to \forall y\, B^{\circ}(x,y)\big).$$

The mixed reading used for administrative checker syntax is not the definition of this ordinary realization.

The inherited arithmetic bridge has four logically ordered stages:

1. Equation induction and numerical cases construct a scalar predecessor P, with internal proofs of $P0 = 0$ and $P(Sn) = n$. Successor injectivity then follows by applying P.
2. Truncated subtraction and selectors yield a characteristic term for actual equality, with a proof of reflection on the entire domain.
3. Every fixed quantifier-free formula is represented by a Boolean equation, recovering contextual quantifier-free induction.
4. Pairing and projections compile every fixed primitive recursion into an initialized scalar iterator. Equation induction proves the defining equations and the uniqueness needed for both round trips.

The full written constructions are reproduced in the attributed appendices. The current repository also contains Lean theorems `biInterp`, `teq_iff_pra`, `pra_iff_teq` and `teq_consistent_iff`, and the newer native theorem `NPrf.iff_teq`. We inspected those sources; the present package does not claim a fresh Lean build or a Lean proof of the new guarded translation.[5, 6]

The translation T of Section 2 changes formulas by computable padding. Inclusion J changes nothing numerical. Their native composites, composed with the inherited ordinary interpretations, give both formula composites. Both numerical composites are internally represented by the identity graph $x = y$. This is stronger than equiconsistency or agreement of standard numerical functions, and it does not use a restriction to standard numerals.

Contradiction in the native calculus is $0 = S0$. The ordinary separation bridge is stated explicitly because positive numerical equations alone admit a singleton structure. Translating finite proofs in both directions preserves contradiction, so the two theories are equiconsistent; no assertion of either theory's self-consistency is involved.

4 Executable checker and its exact theorem

4.1 Statements, certificates, and acceptance

Each compact checker is one standalone Python file importing only the standard JSON parser. Input is a JSON array: the encoded goal is last, and instruction packets precede it in reverse execution order. The driver pops the goal, then the checker pops the instructions it needs. Normal exit with exactly `ACCEPT` is acceptance. An exception, nonzero exit, timeout, or resource exhaustion is not acceptance. Unused prefix packets are permitted, as in the reference protocol.

There are three distinct syntactic levels:

1. The canonical object language is exactly the guarded Eq/D language of Section 2. Closed canonical goals have the integer-and-array encoding below.
2. Administrative proof resources additionally contain Q closures, unguarded atomic paired formulas, and zero/truth aliases. They have the mixed reading proved sound above.
3. Arbitrary JSON input may fail during traversal. A successful use of surplus syntax must have the stated zero or truth meaning; it is never treated as an unconstrained new arithmetic or logical symbol.

The compact kernel checks proofs relative to this decoding contract. It is not itself a parser that rejects every noncanonical end statement. The separately supplied strict profile does enforce a canonical closed goal at its public boundary.

4.2 The inherited arity encoding

The raw shape, rather than a string tag, identifies a constructor. At a formula position, unary arrays represent D, binary arrays equality, and ternary arrays administrative Q. Canonical Q resources use three copies of the body. At a numerical position, unary arrays are successors and ternary arrays are iterators. Zero is encoded by the integer 0 in canonical terms.

Arity	Numerical meaning	Formula meaning
0	zero	$0 = 0$
1	successor	$\mathsf{D}_{x,y}B$
2	zero annotation	equality
3	iterator	administrative $\mathsf{Q}_y B$
≥ 4	zero annotation	$0 = 0$

The first two fields of a ternary formula are annotations; the third is its body. A unary major $[B]$ is exported as $[B, B, B]$ on paired elimination. Tripling any other nonempty formula array produces only a truth alias. This is an inherited implementation device, not a new logical rule.

Bound numerical coordinates have width two. With p protected slots, formula root depth is $2p$, and numerical root depth is $3 + 2p$. Coordinates $0, 1, 2$ denote zero; $3 + 2j$ and $4 + 2j$ name the jth protected slot. Canonical encoding chooses the odd coordinate. Negative coordinates name rigid parameters admitted by the current floor. Successor introduces two dummy zero slots; an iterator introduces one slot, dummy in its counter and initializer and genuine state in its update. Thus traversal can uniformly increase depth by $5 - \text{arity}$.

The opener preserves a scalar in the protected interval and substitutes the supplied numerical image at a relevant unprotected hole. Images are checked at numerical root depth three. Formula-position

scalar padding is truth. Newly allocated negative names are fresh because earlier goals and retained resources were checked at the old floor. Fractional names, if admitted by a compact profile, denote additional fresh parameters and are never silently identified with the designated integer eigenparameter. The full inherited proof and alias tables are in `reference/inherited_arity_checker.tex`.

A canonical atomic guard $I(y, 0; u.h) = I(y, 0; u.k)$ immediately beneath $\mathsf{D}_{x,y}$ has each side encoded as `[5, 0, body]`. The coordinate 5 names the paired universal variable after the iterator's dummy slot. This makes the optional strict goal check a literal structural test; no numerical evaluation or theorem search is used.

4.3 The mathematical change in the source

In the byte reference the continuation image is

```
(m or len(a) % 2 == u) and k
```

Here `u` carries an old introduction choice in the introduction branch. The guarded checker uses

```
(m or not len(a) % 2) and k
```

instead. Among meaningful matrix roots, only equality has even arity. Its introduction therefore retains the progress instance. A paired or administrative closure matrix has odd arity and retains its zero instance. Even surplus roots denote truth, for which induction is valid. The fourth-field witness and the computational initializer protocol are unchanged; a falsy field that previously selected case versus progress no longer makes that choice.

This is not a deletion that preserves arbitrary unguarded native formulas under their old meanings. The guard restriction, the recovered equation induction, and the administrative mixed soundness theorem are what make the change correct.

The complete direct checker is:

```
import json

def c(a, g=[[]], k=0):
    i = lambda a, t=0j, d=0: [i(b, t, d + 5 - len(a)) for b in a] if a * 0 == [] else a if k <= a < d else d % 2
      and i(t, d=3)
    if i(a) in g:
        return
    m, s, u, t = q()
    c(m, g, k)
    a = m in ([s, u], not s and [t] * 3) and i(t, u) == a and i(t, s) or a
    h = m * 3 or (u and [[s and [s], u, t], i(t, [s, u, t], 3) if s else u]) or (a := i(*a, t, 2))
    k -= 1
    c(i(a, [k]), g + [i(h, (m or not len(a) % 2) and k)], k)
q = json.load(open(0)).pop
c(q())
print('ACCEPT')
```

4.4 The two further compact profiles

The byte profile `code/checker_bytes.py` moves the major check into the beginning of the hypothesis-supply expression:

```
h = c(m, g, k) or ...
```

A successful call to `c` always returns `None`. The major check is therefore still mandatory, and the supply expression continues only after success. The earlier endpoint comparison is a pure opener calculation; it consumes no instructions and changes neither the context nor the lexical floor. Moving that calculation before the major check does not change any successful proof. It can change which error occurs first for an unsuccessful certificate, which has no logical effect.

The AST profile uses the inherited lexical loop and the same composition; the token profile uses the inherited global-floor prevalidated elimination protocol. Each profile receives certificates in its own documented protocol. In particular, global and lexical fresh-name supplies are not silently interchanged. The named untrusted producer emits both forms, and the reproduction script selects the appropriate fixture for each program.

4.5 Soundness and canonical completeness

Theorem 4.1 (Checker contract)**.** *For every canonical closed guarded formula A, there is an accepting finite certificate with canonical encoded goal A if and only if $\mathcal{G} \vdash A$, in the ideal unbounded implementation. Every accepting administrative certificate has an iPRA-provable mixed reading. The numerical interpretations are those already proved for $\mathcal{G}$; no theorem producer or external database is trusted.*

Proof. For soundness, induct on a finite accepting call tree. The opener and freshness lemmas are unchanged from the inherited arity proof. Hypothesis closure, numerical transport, closure specialization, paired elimination, and generated computation supplies have the same structural verification as before. An atomic paired formula is now read progressively; its exported Q clause follows by equation induction in iPRA. Compound paired formulas keep their ordinary reading.

In an introduction continuation, the parity test now supplies exactly the root-determined premise. Atomic progress is sound directly for its progressive administrative reading. Compound case introduction is sound using numerical coverage. Truth aliases remain harmless. A computation hypothesis with only a known zero instance is discharged by substituting zero for its fresh parameters and cutting the computation proof, exactly as in the inherited zero-supply lemma. The mandatory major verification and the profile's scope discipline are preserved.

For canonical guarded A, the mixed reading is provably equivalent in iPRA to its uniform ordinary reading. The explicit bi-interpretation of $\mathcal{G}$ therefore converts the accepted proof back into a guarded proof of A. This last step is why the soundness theorem remains valid even if a certificate uses unguarded intermediate resources.

Conversely, encode a finite guarded proof with fresh integer names. Use the inherited packets for computation, transport, clauses and elimination. At introduction, a canonical atomic matrix always uses progress and a compound matrix always uses cases, exactly matching the new parity test. A restricted computation leaf is repaired as a standalone equation, preserving its original conclusion by transport; this does not replace the literal zero initializer inside a canonical paired guard. When the dispatch interface requires a nonempty witness, use the computation-backed box $I(0, t; v.0) = t$ for the existential substitution parameter. This preserves the matrix's fixed guard syntax. Thus every canonical guarded proof compiles to a finite accepting certificate. □

This theorem is deliberately not a full formula-composite claim for every administrative array. The unrestricted raw atomic D calculus has two sound standard-number readings that disagree on unguarded matrices. For example, with unused x and $E(y) \equiv y = 1$, its old conditional reading is true while its progressive reading fails at $1 \to 2$. A formula composite through the guarded embedding

identifying this unrestricted binder with its progressive reading would contradict that model. No impossibility result for every conceivable interpretation of the administrative theory is asserted.

4.6 Explicit canonical-goal validation and its cost

`code/strict_checker.py` is also standalone. It validates a closed integer-and-array Eq/D goal, admits only the canonical odd bound coordinates and literal zero, and checks that every atomic paired matrix has both sides prefixed by `[5, 0]`. It then runs the complete direct kernel. It does not restrict administrative intermediate resources, whose soundness is proved separately. This validates the stated canonical arity convention, not a unique raw spelling: an allowed odd coordinate may name a dummy zero slot.

For comparison, `reference/baseline_strict_goal.py` is a derivative we supply here: the old byte kernel with the same closed-goal traversal, but without the new atomic guard restriction. It is not an upstream frontier artifact. These additional whole-module costs are:

Additional whole module	Bytes	Tokens	AST	Gzip 9
Readable guarded checker	1275	292	336	511
Guarded strict-goal checker	1208	491	568	574
Current core plus strict goal parser	1109	450	521	538

Thus a contract that explicitly rejects all noncanonical end statements does not inherit the compact byte advantage. Both the validator and its cost are visible. The primary comparison is exactly the repository's canonical-encoding convention, with the new convention stated in full.

4.7 Executable evidence

The delivered verification ran 800 command-line checks across 10 whole programs, in ordinary and optimized Python, with 0 mismatches. There are 8 canonical named examples, including a guarded induction derivation whose hypothesis is used, together with an administrative example and an old atomic case theorem that the deterministic checker rejects.

The canonical examples exercise both computation equations, numerical transport, guarded introduction and elimination, implication between equations, and compound case introduction. The nontrivial guarded induction example compiles to 176 instructions in the lexical protocol; its progress hypothesis is used through the compound implication and case construction from the proof of equation induction. The named-proof audit records 13 atomic paired occurrences in that example, all guarded.

The inherited negative controls check false conclusions, incorrect computation endpoints, escaping names, leaked contexts, missing packet fields, and forbidden compound progress. One inherited control changes only the retired case/progress field; it is intentionally accepted by the deterministic checker when its atomic progress proof is otherwise valid. The receipt labels this behavioral change explicitly. All other expected rejections and the old unguarded case counterexample are recorded separately.

These checks establish that the supplied programs and certificates behave as reported. They do not replace the universal soundness, completeness, and interpretation proofs above. Physical stack and memory limits can reject a finite proof that would be accepted in the ideal implementation, as for the reference checkers; resource exhaustion never proves a theorem.

5 Novelty, limits, and reproduction

The current papers already contain the one-binder essential language, padding gates, the Choice-DI equivalence, witness-dependent introduction, progressive-binder alternatives, zero-supply normalization, and the arity codec. None of those is claimed as new here. The new combination is the guarded canonical language with root-determined introduction, the predecessor-free recovery of full equation induction, the two explicit native composites, and the use of the mixed administrative reading to justify a smaller checker without an uncharged guard test. The source inspection did not find this guarded restriction-and-translation theorem in the pinned principal reports; that is a comparison with this repository, not a literature-wide priority claim.

The result has three material limitations. First, the measured gain is small. Second, under a different input contract requiring an explicit validator for canonical goals, that validator must be charged, and the compact frontier figures cannot be quoted for it. Third, the new universal metatheorems are written mathematics; process tests exercise concrete certificates and rejected attacks but do not formally verify the proofs for all inputs.

The archive contains the complete report source and PDF, the four compact profiles, a readable profile, a standalone strict-goal profile, frozen baseline programs, the untrusted named proof producer, positive and negative certificates, source hashes, measurements and the independent mathematical review. The proof generators and measurement scripts are not imported by any checker.

From the archive directory, run:

```
python3 scripts/reproduce.py
python3 code/checker.py < tests/root_cases/guarded_literal_equation_introduction_local_neutral.json
latexmk -pdf -interaction=nonstopmode -halt-on-error report.tex
```

The reproduction script reruns the certificate checks in ordinary Python and with `-O`, remeasures each whole module, and writes fresh machine-readable receipts. Python and zlib versions can change byte normalization or compression figures; the supplied receipt records the environment used for this delivery.

Attribution and inherited proofs

The following five sections are reproduced from the pinned ProveIt `Research/certified_zero/reference/` files under the repository's MIT No Attribution license. They establish the inherited Choice-DI logical basis and the scalar arithmetic bridge used above. They are included for a self-contained mathematical package and are not new results of this report. Historical wording such as "new" or "current" inside these reproduced sections refers to their source reports. Their complete original source files and license are retained in `reference/`. The separate arity-checker proof is supplied there as `inherited_arity_checker.tex`; its old introduction-choice step is replaced in this report by the deterministic-rule and administrative soundness argument.

A Complementary introductions with one local assumption

The old paired introduction retains both the zero instance and an induction instance when its matrix is an equation. Both together are unnecessary. A certificate may choose either one, provided the induction choice is restricted to a literal equation. This changes neither the formula language nor the meaning of a formula. The omitted combination is recovered by a short nested introduction–elimination construction.

A.1 Complete native syntax and rule inventory

The scalar numerical language is

$$t ::= x \mid 0 \mid St \mid I(n, a; u.h).$$

Only h binds the state variable u; n and a do not. The update may contain fixed numerical parameters. The computation axioms in every retained context are

$$I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u]. \tag{I0, IS}$$

The argument also applies unchanged to the full fixed-description PR language with its projection, composition, and recursion equations. No numerical variable ranges over function descriptions, and no uniform evaluator is postulated.

Native formulas and reusable context clauses are

$$A ::= (s = t) \mid \mathsf{D}_{x,y}A, \qquad \langle A; y.B(y)\rangle, \quad y \notin \mathrm{FV}(A).$$

Bound variables are renamed before capture-free substitution. The two variables x, y in a paired formula are distinct. An ordinary assumption F is the constant clause $\langle H; v.F\rangle$, where v is absent from F and H is the fixed computation theorem $I(0, 0; u.0) = 0$. Thus ordinary hypothesis use is a clause Use preceded by this computation proof.

There are computation axioms and exactly the following inference patterns:

$$\frac{\Gamma \vdash A}{\Gamma \vdash B(t)}\ \mathsf{Use} \quad (\langle A; y.B(y)\rangle \in \Gamma),$$

$$\frac{\Gamma \vdash \mathsf{D}_{x,y}B(x, y) \qquad \Gamma, \langle B(a, 0); y.B(a, y)\rangle \vdash F}{\Gamma \vdash F}\ \mathsf{DE}, \qquad \frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)}\ \mathsf{Tr},$$

$$\frac{\Gamma, B(t, \eta_\varepsilon(z)) \vdash B(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y}B(x, y)}\ \mathsf{DI}_\varepsilon, \qquad \eta_0(z) = 0, \quad \eta_1(z) = z. \tag{Choice DI}$$

Here ε is a certificate choice in $\{0,1\}$, and $\varepsilon = 1$ is permitted only when B has a literal equation root. An equation beneath a binder is not a literal equation. Call $\varepsilon = 0$ the *case choice* and $\varepsilon = 1$ the *progress choice*. Both retain exactly one ordinary formula assumption. Compound matrices have only the case choice.

In DI the eigenparameter z is fresh for Γ, the witness t, and every fixed parameter of B. In DE the eigenparameter a is fresh for Γ, the conclusion F, and every fixed parameter of B. In transport, $F(v)$ is any fixed native formula template. Displayed local assumptions and clauses are discharged. There are no primitive reflexivity, coverage, separation, ordinary logical, or additional induction rules.

We call this calculus $\mathcal{C}$. The original comparison calculus $\mathcal{N}$ has identical syntax, computation, Use, DE, and transport, but its introduction is

$$\frac{\Gamma, B(t,0), [B(t,z)]_{\mathrm{at}} \vdash B(t,Sz)}{\Gamma \vdash \mathsf{D}_{x,y} B(x,y)}. \tag{Old DI}$$

The bracket supplies $B(t,z)$ exactly at a literal equation.

A.2 Exact equivalence and the unused-witness construction

Lemma A.1 (Structural admissibility)**.** *Weakening, exchange, contraction, renaming, ordinary-assumption cut, and capture-free numerical substitution are admissible in $\mathcal{C}$.*

Proof. Induct on derivations. Rename local eigenparameters before inserting proofs or terms. Numerical substitution preserves a formula's root constructor, so an allowed progress choice remains allowed. The selected choice does not change under substitution or cut. For ordinary-assumption cut, replace each use of its constant clause by the supplied proof. Computation schemas are closed under capture-free numerical substitution. These facts verify every case. □

Theorem A.2 (Identity-language equivalence)**.** *For every retained clause context Γ and native formula F,*

$$\Gamma \vdash_{\mathcal{C}} F \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{N}} F.$$

The transformations are effective on finite derivations and leave all numerical and formula syntax unchanged.

Proof. Each new introduction is an instance of the old one after weakening by the omitted local assumption. For the case choice at an equation, add the induction instance. For the progress choice, add the zero instance. Compound case introduction is already the old rule. Thus every new derivation translates to an old one.

For the reverse direction, transform an old derivation recursively. Only its atomic DI case needs a construction. Suppose its matrix is the equation $E(x,y)$ and its transformed premise is

$$\pi: \quad \Gamma, E(t,0), E(t,z) \vdash E(t,Sz).$$

The outer new inference will be case DI with witness t. It is therefore enough to prove $E(t,Sz)$ under $\Gamma, E(t,0)$.

Rename the eigenvariable in π to a fresh v. Choose a new variable u absent from $E(t,y)$ and all retained data. Under $\Gamma, E(t,0)$, progress DI with witness zero introduces

$$\mathsf{D}_{u,y} E(t,y)$$

from exactly the renamed premise $\pi[v/z]$. The bound coordinate u is deliberately unused.

Apply DE to this paired formula. Because u is unused, the opened clause is exactly

$$\langle E(t,0); y.E(t,y)\rangle;$$

it contains no new existential witness. The retained ordinary assumption $E(t,0)$ supplies its antecedent. Use at Sz gives $E(t,Sz)$, and DE closes. Finally the outer case DI introduces the original $\mathsf{D}_{x,y}E(x,y)$ with witness t. The original premise proof π has been inserted once.

An old compound introduction already is the new case choice. All other rule cases are unchanged. This completes the induction on the original derivation. □

The unused inner coordinate is essential. Introducing $\mathsf{D}_{x,y}E(x,y)$ internally and then eliminating it would give an arbitrary fresh witness, not necessarily the original t. The construction instead keeps t as a fixed parameter of $E(t,y)$ and binds only the unused coordinate u.

Proposition A.3 (Linear bound on inference nodes)**.** *Count each computation axiom, Use, DI, DE, and transport as one inference node, expanding ordinary-assumption use as a computation of H followed by Use. If an old derivation has N nodes and r atomic DI nodes, the displayed transformation has exactly*

$$N + 5r \le 6N$$

nodes.

Proof. Each atomic old DI node is replaced by six nodes: outer case DI, inner progress DI, DE, Use of the opened clause, Use of the retained base assumption, and the computation of H needed for that last ordinary-assumption use. Its old premise subtree occurs once. All other inference nodes are copied once. □

This counts inference-tree nodes. It is not a bound on serialized JSON bytes, formula sizes, checker runtime, or the number of consumed packet cells when hypothesis closure is implicit. In the formula-context Q/D presentation, the corresponding replacement has outer case DI, inner progress DI, DE, QE, and two Hyp nodes, giving the same six-node count.

A.3 Induction and numerical cases separate cleanly

Lemma A.4 (Contextual equation induction)**.** *The progress choice, DE, and Use derive*

$$\frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)} \tag{EI}$$

for every equation E, with the usual freshness condition.

Proof. Choose an unused witness coordinate u. Progress DI introduces $\mathsf{D}_{u,y}E(y)$ from the given step. DE opens the clause $\langle E(0); y.E(y)\rangle$. Supply its antecedent using the given base proof and use the clause at t. Close DE. □

Lemma A.5 (Cases, equality, and binder congruence)**.** *The case choice derives, for arbitrary native formulas,*

$$\frac{\Gamma \vdash F(0) \qquad \Gamma \vdash F(Sz)}{\Gamma \vdash F(t)} \tag{NC}$$

with the usual freshness condition. Equality reflexivity, symmetry, transitivity, numerical congruence, and uniform matrix interderivability under D are derivable without using the progress choice.

Proof. For cases, use case DI with an unused witness coordinate and matrix $F(y)$, ignoring its retained base in the successor premise. DE opens $\langle F(0); y.F(y)\rangle$; the given zero proof and Use at t finish.

For reflexivity put $r = I(0, t; u.0)$. Use the computation equation $r = t$ as both transport premises with motive $v = t$ to obtain $t = t$. The usual explicit equality motives give symmetry, transitivity, and numerical congruence.

For binder congruence, open the old paired formula at a fresh witness a and introduce the new one by case DI at a. Convert its retained base to the old base, use the opened old clause at the successor, and convert back. Ordinary cut closes the construction, and the reverse conversion gives the converse. No induction is used. □

The primitive progress choice is still restricted to equations. Allowing it at arbitrary compound matrices would, by the argument for (EI), supply induction for arbitrary formulas and change the intended theory.

A.4 The entire ordinary logical interface

Here is a predecessor-free reconstruction of the ordinary logical operations. It makes explicit that the case choice supplies the logical interface independently of progress induction.

Let $C(v, a, b) := I(v, a; u.b)$ with u unused. Computation gives $C(0, a, b) = a$ and $C(Sv, a, b) = b$. Define the gate on fixed finite formula syntax by

$$G_v(s = t) := (C(v, 0, s) = C(v, 0, t)), \qquad G_v(\mathsf{D}_{x,y}B) := \mathsf{D}_{x,y}G_vB,$$

renaming bound variables away from the control. Equality, binder congruence, and external structural induction give

$$\vdash G_0A, \qquad G_{Sv}A \dashv\vdash A, \qquad A \dashv\vdash \mathsf{D}_{u,y}G_yA \quad (u, y \notin \mathrm{FV}(A)). \tag{Gate and padding}$$

For padding, case DI proves the forward direction. DE, the zero-gate theorem, and Use at 1 prove the reverse direction.

Define a selector between two arbitrary formulas by

$$M_v(s = t, u = w) := (C(v, s, u) = C(v, t, w))$$

and, with aligned fresh paired variables, by

$$\begin{aligned} M_v(\mathsf{D}_{x,y}A, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(A, B),\\ M_v(E, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(G_yE, B),\\ M_v(\mathsf{D}_{x,y}A, E) &:= \mathsf{D}_{x,y}M_v(A, G_yE), \end{aligned}$$

where E is an equation. Every call decreases maximum input binder depth. Padding and congruence prove $M_0(A, B) \dashv\vdash A$ and $M_{Sv}(A, B) \dashv\vdash B$.

Put $\delta(v) = C(v, 1, 0)$ and $p(z) = I(z, 0; u.\delta(u))$. Only the fixed computations $p(1) = 1$ and $p(2) = 0$ are needed. The full logical macro inventory is

$$\begin{aligned} A \Rightarrow_* B &:= \mathsf{D}_{u,z}M_z(A, B),\\ A \wedge_* B &:= \mathsf{D}_{u,z}G_zM_{p(z)}(A, B),\\ \exists_* x\, A(x) &:= \mathsf{D}_{x,y}G_yA(x),\\ A \vee_* B &:= \exists_* x\, M_x(A, B),\\ \forall_* x\, A(x) &:= A(0) \wedge_* \mathsf{D}_{u,z}G_zA(z),\\ \bot_* &:= (0 = 1). \end{aligned} \tag{Logical macros}$$

Newly bound variables are fresh; u is unused.

Implication introduction obtains its assumption from the zero selector and supplies the consequent at successors, then uses case DI. Elimination opens its clause and uses it at 1. Existential introduction uses its witness and the successor gate; elimination opens at a fresh witness, supplies the zero-gate theorem, and uses the clause at 1.

Disjunction introduction uses witnesses 0 and 1. For elimination, discharge the two branch proofs as $A \Rightarrow_* F$ and $B \Rightarrow_* F$. NC applied to $M_z(A, B) \Rightarrow_* F$ gives this implication at every control: the two endpoint cases follow from the branch implications. Existential elimination provides $M_a(A, B)$ and the implication gives F.

Given A and B, selector endpoints and NC give $M_v(A, B)$ at an arbitrary control. Substitute $v = p(Sz)$ and use the successor gate for conjunction introduction. For projections, open its zero-gated clause and specialize at 1 and 2; the two fixed computations give B and A, respectively.

These rules prove existential numerical coverage:

$$(x = 0) \vee_* \exists_* v\, (x = Sv). \tag{Coverage}$$

Use NC. Zero takes the left injection of reflexivity, and a successor takes the right injection with the evident witness. Coverage gives equality cases even when the retained context contains x; its elimination adds $x = 0$ or $x = Sv$ rather than silently changing that retained context.

Universal introduction follows from a fresh-parameter proof by substituting zero and successors and forming the two conjuncts. For elimination, take its $A(0)$ conjunct and open the paired conjunct. Its base is a zero-gate theorem. Coverage of the target chooses the zero instance from $A(0)$ or a successor instance from the opened clause and gate removal.

Finally, under $0 = 1$, computation and transport give $s = C(0, s, t) = C(1, s, t) = t$ for arbitrary numerical terms. Every equation follows, and external induction on formula syntax with case DI gives every native formula. This supplies falsity elimination. No excluded middle for quantified formulas has entered the construction.

A.5 Whole-domain realizations and both formula composites

For a standard first-order reading, let T_{eq} have intuitionistic equality logic, the same numerical signature and computation equations, numerical coverage, separation $(0 = 1) \to \bot$, and contextual equation induction. Its interpretation of the native syntax is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y} B)^\circ = \exists x (B^\circ(x, 0) \to \forall y B^\circ(x, y)),$$

$$\langle A; y.B \rangle^\circ = A^\circ \to \forall y B^\circ.$$

The domain is the entire numerical domain and interpreted equality is actual equality.

Case DI is sound in this realization by numerical coverage. Progress DI is sound by contextual equation induction under the conditional base. Use, DE, transport, and computation have their displayed ordinary meanings. Conversely, the native macros derive every ordinary logical rule, (EI) derives equation induction, and (Coverage) derives coverage. Separation translates to the identity implication $(0 = 1) \Rightarrow_* (0 = 1)$.

For the essential paired formula composite,

$$\mathsf{D}_{x,y} B \dashv\vdash \exists_* x (B(x, 0) \Rightarrow_* \forall_* y\, B(x, y)). \tag{Paired composite}$$

Forwards, DE opens its clause at a fresh a; under $B(a,0)$ that clause supplies $B(a,y)$ at a fresh arbitrary y. Introduce the universal, implication, and existential macros and close DE. Backwards, eliminate the existential at a fresh a. Introduce the paired formula with witness a by case DI: the retained base, supplied implication, and universal elimination at Sz give its successor premise.

The ordinary interpretations of the six logical macros have their ordinary round trips. For conjunction, its true base and the instances $1, 2$ recover both operands; the converse uses selector cases. The universal has all successor instances in its paired conjunct and zero in its first conjunct; coverage gives exactly the ordinary universal. Implication, disjunction, and existential quantification use their selector or gate endpoints. Falsity uses the explicit ordinary separation bridge. External induction on finite formulas, together with (Paired composite), gives both formula composites. Clauses correspond in both directions to $A \Rightarrow_* \forall_* yB$ by the derived logical rules.

The numerical composites are literal identities with internally provable graph $x = y$. No standard-numeral restriction, retract, quotient, or equality reflection assumption is being inserted at this stage. The separately supplied arithmetic bridge identifies T_{eq} with full iPRA; the present change preserves that bridge unchanged because Theorem A.2 preserves every native sequent.

A.6 What the theorem does and does not claim

The new inference interface retains one ordinary assumption at every paired introduction. It preserves the entire consequence relation, not just equations or closed theorems. Consequently the established full-iPRA interpretations and equiconsistency transfer unchanged.

This result is different from retaining only progress introduction and strengthening closure elimination. Here the original elimination rule is retained, and the case and progress introductions coexist. It is also different from allowing arbitrary compound progress, which would supply stronger induction.

A complete-checker comparison must still count the certificate choice, its equation-root restriction, substitution, context handling, computation, and the public input/output interface. The mathematical theorem alone is not a byte-count or minimality theorem. The inference-node bound is distinct from source-size measurements and from serialized certificate size. The proof is a written universal proof; finite regression tests do not constitute a Lean/Rocq formalization of it.

B Arithmetic strength and both ordinary interpretation composites

We spell out the arithmetic bridge to avoid identifying standard numerical agreement with bi-interpretability. First elaborate every fixed iterator into an ordinary function symbol. At stage zero the symbols are $0, S$. Given a fixed earlier-stage term $h(u, \vec{p})$, introduce $i_h(n, a, \vec{p})$ with the two equations

$$i_h(0, a, \vec{p}) = a, \qquad i_h(Sn, a, \vec{p}) = h(i_h(n, a, \vec{p}), \vec{p}).$$

Only finite descriptions occur; the domain contains no code for a universal evaluator. Formal parameters are distinct when a description is declared, while actual arguments can coincide.

Let T_I be ordinary intuitionistic first-order logic with actual equality and genuine falsity, these staged equations, contextual induction for literal equations, and

$$\forall x\,(x = 0 \vee \exists y\, x = Sy), \qquad (0 = S0) \to \bot.$$

The second sentence is the falsity bridge. It must be stated: positive numerical equations by themselves

admit a singleton structure. Native explosion from $0 = S0$ and a translation of ordinary falsity to that equation do not authorize silently dropping the bridge from T_I.

The native-to-ordinary translation is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y}B)^\circ = \exists x\big(B^\circ(x, 0) \to \forall y B^\circ(x, y)\big).$$

A clause $\langle A; y.B\rangle$ becomes $A^\circ \to \forall y B^\circ$. For the auxiliary closure used by the compact checkers, $(\mathsf{Q}_y B)^\circ = B^\circ(0) \to \forall y B^\circ$. The interpretation domain is all elements, defined by $x = x$; equality is actual equality.

Case DI is sound because numerical coverage combines its retained base with its successor proof. Progress DI is sound by literal-equation induction under a temporarily assumed base. Use, DE and transport are ordinary logical inferences under the translations. Conversely the derived logical operations, equation induction, cases and explosion in Section A recover the rules and axioms of T_I. An auxiliary closure is interderivable with $\mathsf{D}_{v,y}B(y)$ for unused v: in one direction use case DI with the closure and its base; in the other eliminate the paired formula and return its supplied closure. Thus formulas and context clauses have the stated meanings in both presentations.

B.1 The arithmetic dependency order

The scalar update has no recursion-counter argument. We do not conceal one in an encoding. The inherited internal proofs, reproduced in the appendices, establish the required arithmetic in this order:

1. Using equation induction and numerical cases, construct a fixed scalar predecessor term and prove $P0 = 0$ and $P(Sn) = n$. Existential coverage then gives $x = 0 \vee x = S(Px)$: in the successor case $x = Sy$, apply P to obtain $Px = y$ and substitute.
2. Construct truncated subtraction and a numerical characteristic of actual equality. Prove its reflection on the entire original domain.
3. Compile each fixed quantifier-free formula into its characteristic equation and recover every contextual quantifier-free induction instance.
4. With that induction available, construct pairing and projections by a bounded square-root search and compile every fixed general primitive-recursive description into a scalar iterator term.

No pairing or faithful equality detector is assumed to obtain predecessor. No appeal to standard-model correctness replaces an internal equality. Every induction fixes its passive parameters; diagonal substitution occurs only into a previously proved identity.

For a target recursion

$$f(0, \vec{p}) = g(\vec{p}), \qquad f(Sn, \vec{p}) = h(n, f(n, \vec{p}), \vec{p}),$$

let π, L, H be the proved pairing and projections. The compiler uses

$$V(n, \vec{p}) = I\big(n, \pi(0, g^*(\vec{p})); u.\pi(SL(u), h^*(L(u), H(u), \vec{p}))\big), \qquad f^*(n, \vec{p}) = H(V(n, \vec{p})).$$

Equation induction proves $L(V(n, \vec{p})) = n$; the two projection identities then prove the target defining equations. The construction is external recursion on each finite PR description, not an internal quantification over arbitrary functions.

Theorem B.1 (Whole-domain bi-interpretability and equiconsistency)**.** *The ordinary realization T_I and full first-order iPRA are bi-interpretable. In both theories $x = y$ defines the isomorphism from the original domain to its double interpretation. The native Choice-DI calculus is therefore a faithful proof presentation of a theory bi-interpretable and equiconsistent with iPRA.*

Proof. Interpret an iterator in iPRA by primitive recursion that ignores its counter argument in the update. iPRA proves its equations, literal equation induction, coverage and separation. In the reverse direction use the scalar compiler. The predecessor and equality-reflection proofs supply the successor axioms and every quantifier-free induction instance. All ordinary logical rules are preserved.

For two iterators with the same seed and provably equal updates, equation induction on the equality of their values proves uniqueness at every counter, with all parameters fixed. The corresponding argument applies to ordinary primitive recursion. Induct externally on a finite description: its double translation satisfies its original defining equations, so internal uniqueness identifies the two functions. This also proves coherence when substitution changes the staged syntactic description of an update.

The domain and equality are unchanged; hence the identity graph is internally total, bijective and preserves every interpreted operation. The displayed ordinary meanings of the logical macros and the paired binder give both formula composites by external induction on finite formula syntax. Clauses are similarly equivalent to their conditional universal meanings. The explicit falsity bridge preserves contradiction in both directions. Effective translations of finite proofs give equiconsistency, without a claim that either theory proves its own consistency. □

C Internal scalar predecessor bootstrap

This scalar bootstrap is inherited from the supplied scalar development [11] and is reproduced to keep the scalar checker self-contained. It is not a new contribution of this paper. The proof uses a factor map fixing odd states and a saturated-iteration construction of predecessor.

The available arithmetic terms are 0, S, and

$$I(n, a; u.h), \qquad I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u].$$

The update binds only u. All other variables in it are fixed numerical parameters. We use equality reasoning, the already reconstructed zero/successor cases, and induction on a single literal equation. Every induction below keeps its other parameters fixed. Predecessor, successor injectivity, pairing, and induction on arbitrary quantifier-free formulas are not assumptions of the argument.

C.1 Elementary terms and two finite cycles

Write $1 = S0$ and use larger numerals as successor abbreviations. Define

$$C(n, a, b) = I(n, a; u.b), \qquad Z(n) = C(n, 1, 0), \qquad a + b = I(b, a; u.Su).$$

The update variable of C is absent from its body. Consequently

$$C(0, a, b) = a, \qquad C(Sn, a, b) = b.$$

Numerical cases show that $Z(n)$ is 0 or 1. The usual commutative-monoid equations for addition follow by literal equation induction. For example, $0 + b = b$ and $Sa + b = S(a + b)$ are proved by induction on b; these yield commutativity by a further equation induction, and associativity follows by induction on the last summand. We use these equations to reorder finite sums. The notation $2a$ means $a + a$ and $3a$ means $a + a + a$; no multiplication operation is needed for this bootstrap.

Put

$$N(n) = I(n, 0; u.Z(u)), \qquad j(u) = Z(u) + 2N(u), \qquad M(n) = I(n, 0; u.j(u)).$$

The range of N is contained in $\{0,1\}$: its zero value is 0, and its successor value is a value of Z. This uses numerical cases on its counter, not induction on a disjunction. Hence

$$N(n) + Z(N(n)) = 1, \qquad Z(Z(N(n))) = N(n), \qquad N(n+2) = N(n). \tag{1}$$

Here the last equation follows from two successor computations and the preceding equation. Literal-equation inductions give

$$N(2n) = 0, \qquad N(2n+1) = 1, \qquad N(3n) = N(n). \tag{2}$$

For the first equation the successor step adds two to the argument of N; the second follows by one computation. In the third, increasing n adds three to the left argument and therefore applies Z once to both sides of the induction equation.

Finite computations now give

$$j(0) = 1, \qquad j(1) = 2, \qquad j(2) = 0.$$

The proof below needs just these three values and $M(Sn) = j(M(n))$; it does not assume a division algorithm or a theorem characterizing M as remainder modulo three.

C.2 A ramp with a global scaling identity

Define

$$G(n) = I(n, 0; u.u + M(u) + 1).$$

The central invariant is

$$M(G(n)) = N(n). \tag{3}$$

Its zero case is immediate. For its successor step, retain the single induction equation and split the Boolean value $N(n)$. If it is 0, the next G state is $G(n) + 1$; its M value is $j(0) = 1$, equal to $N(Sn)$. If it is 1, the next G state is $G(n) + 2$; two computations of M change its value from 1 to 2 and then to 0, again equal to $N(Sn)$. This proves the invariant by literal-equation induction.

It follows that

$$G(Sn) = G(n) + N(n) + 1, \qquad G(n+2) = G(n) + 3.$$

Equation induction therefore yields

$$G(2n) = 3n. \tag{4}$$

A separate literal-equation induction proves

$$G(3n) = 3G(n) + N(n). \tag{5}$$

For completeness, its successor calculation is

$$\begin{aligned} G(3Sn) &= G(3n) + N(3n) + 4 = 3G(n) + 2N(n) + 4, \\ 3G(Sn) + N(Sn) &= 3G(n) + 3N(n) + 3 + Z(N(n)). \end{aligned}$$

The expressions agree in both Boolean cases for $N(n)$. The base is $0 = 0$. In particular, (5) was proved internally; it was not inferred from an intended floor-function interpretation of G.

Set

$$H(u) = C(N(u), G(u), u).$$

The crucial choice is to retain u when its parity is 1. The preceding equations give

$$H(2u) = 3u, \qquad H(1) = 1, \qquad H(3u) = 3H(u). \tag{6}$$

To check the last identity, use $N(3u) = N(u)$ and split the Boolean $N(u)$. At 0, both sides are $3G(u)$ by (5); at 1, both are $3u$. This argument does not require that an arbitrary number has already been represented as $2v$ or $2v + 1$.

C.3 A comparator from scalar iteration

For every fixed update h, equation induction on n, keeping m, a and all parameters fixed, proves the semigroup identity

$$I(m + n, a; h) = I(n, I(m, a; h); h).$$

The base is the zero computation and the step applies h to the induction equation. This identity will also be used in the later pairing construction.

For a fixed update h, equation induction on y, with its initializer a and all parameters of h fixed, proves the rotation identity

$$I(Sy, a; h) = I(y, h(a); h). \tag{7}$$

Both sides have base $h(a)$, and their successor equations apply the same h to the previous equality. Here and below $I(y, a; h)$ is an abbreviation for one fixed finite scalar term, not quantification over functions.

By (6), another equation induction gives

$$I(y, 3a; H) = 3I(y, a; H). \tag{8}$$

Its base is $3a = 3a$. In the step the fixed-parameter induction equation rewrites the new left side as

$$H(3I(y, a; H)) = 3H(I(y, a; H)).$$

The initializer parameter stays a throughout this induction. Equation induction also gives $I(y, 1; H) = 1$, since $H(1) = 1$.

Define

$$E(x) = I(x, 1; u.2u), \qquad L(x, y) = N\big(I(y, E(x); H)\big).$$

Then the following entire recursion specification is provable:

$$L(0, y) = 1, \qquad L(Sx, 0) = 0, \qquad L(Sx, Sy) = L(x, y). \tag{9}$$

The first equation uses $E(0) = 1$ and the fixed-point identity for H. The second uses $E(Sx) = 2E(x)$ and $N(2a) = 0$. For the third,

$$\begin{aligned} L(Sx, Sy) &= N\big(I(Sy, 2E(x); H)\big) \\ &= N\big(I(y, H(2E(x)); H)\big) \\ &= N\big(I(y, 3E(x); H)\big) \\ &= N\big(3I(y, E(x); H)\big) \\ &= N\big(I(y, E(x); H)\big) = L(x, y). \end{aligned}$$

The successive steps use rotation, $H(2a) = 3a$, iterator scaling, and $N(3a) = N(a)$. Thus L has the comparison equations at all arguments internally. Its standard-model meaning as the indicator of $x \leq y$ is motivation, not a premise of the following argument.

C.4 Saturated iteration gives predecessor

We now need only the three equations (9). Define

$$B_m(u) = C(L(Su, m), u, Su), \qquad W(t, m) = I(t, 0; u.B_m(u)).$$

The subscript in B_m records a fixed parameter; B_m is an open term abbreviation, not a function variable.

Numerical cases on e prove the selector equation

$$C(e, Sa, Sb) = SC(e, a, b).$$

Equivalently one may use equation induction on e, whose induction hypothesis is unused. Therefore

$$\begin{aligned} B_{Sm}(Su) &= C(L(SSu, Sm), Su, SSu) \\ &= C(L(Su, m), Su, SSu) = SB_m(u), \qquad (10) \\ B_{Sm}(0) &= C(L(S0, Sm), 0, S0) = C(L(0, m), 0, S0) = S0. \qquad (11) \end{aligned}$$

Literal-equation induction on t, with m fixed, proves

$$W(St, Sm) = SW(t, m). \qquad (12)$$

The base is (11). The successor step is

$$\begin{aligned} W(SSt, Sm) &= B_{Sm}(W(St, Sm)) \\ &= B_{Sm}(SW(t, m)) \\ &= SB_m(W(t, m)) = SW(St, m). \end{aligned}$$

Only the second line uses the current induction hypothesis, at the same fixed m.

Two subsequent equation inductions give

$$L(Sn, n) = 0, \qquad W(n, n) = n. \qquad (13)$$

The first uses the comparator recurrence. The second uses (12) in its successor step. The substitution $t = m = n$ is made into that already proved universally parameterized equation; no parameter of an active induction hypothesis is changed. Consequently

$$W(Sn, n) = B_n(n) = C(L(Sn, n), n, Sn) = n. \qquad (14)$$

Finally define the finite scalar term

$$P(m) = I\big(m, 0; u.C(L(SSu, m), u, Su)\big). \qquad (15)$$

Its zero equation is immediate. To justify its successor equation without assuming a separate extensionality rule for iterator bodies, put

$$V(t, m) = I\big(t, 0; u.C(L(SSu, Sm), u, Su)\big).$$

The comparator recurrence identifies this update pointwise with $B_m(u)$. Equation induction on t, with m fixed, gives $V(t, m) = W(t, m)$: their base values are 0, and the successor step uses the pointwise update equation and then the induction equation. Now (14) yields

$$\boxed{P(0) = 0, \qquad P(Sm) = V(Sm, m) = W(Sm, m) = m.}$$

C.5 Dependency boundary

All terms in this section are built from $0, S, I$ by finite substitution. All inductions have a single equation as their invariant. Numerical cases and equality transport are the already reconstructed logical resources. Successor injectivity may now be obtained by applying P to an assumed equation $Sx = Sy$. Equality tests, reflection of quantifier-free formulas, pairing, and the recovery of general primitive recursion can then be developed in the established order. They were not used in order to manufacture predecessor.

This argument is a written internal derivation in the stated theory. The finite implementation checks do not constitute a proof-assistant formalization of this entire metatheorem.

D From literal-equation induction to full intuitionistic PRA

This section supplies an unconditional completion theorem. It is adapted with attribution from the current forward-left draft [7]. In the scalar application used here, every function occurring in the proof is already a scalar term: predecessor is supplied by Appendix C, the selector is $C(n, a, b) = I(n, a; u.b)$, addition is $x + y = I(y, x; u.Su)$, and truncated subtraction is $d(x, y) = I(y, x; u.Pu)$. All other functions below are finite compositions of these. Thus this proof does not presuppose that arbitrary primitive recursion has already been compiled. The QFI conclusion first applies to the scalar language; the general PR compiler comes afterward. Literal-equation induction, together with the primitive-recursive defining equations, predecessor coverage, and separation of 0 and 1, proves induction for every quantifier-free formula. In particular, the proof does not assume a faithful numerical equality test, stability or decidability of equality, successor uniqueness, or any commutation law for an iterator. We construct the equality test and prove its reflection property for arbitrary elements of the numerical domain.

D.1 The exact hypotheses

Work in ordinary intuitionistic first-order logic with equality and genuine falsity $\bot$. The language has one numerical sort and a symbol for each fixed, finite primitive-recursive description. Distinct descriptions are not identified merely because they compute the same standard function. Let PRE contain the defining equations for zero, successor, projections, composition, and primitive recursion. In particular, choose descriptions P and C with equations

$$P(0) = 0, \qquad P(Sx) = x, \tag{16}$$

$$C(0, a, b) = a, \qquad C(Sx, a, b) = b. \tag{17}$$

Write $1 = S0$. In addition to PRE, assume

$$\mathsf{Cov}: \quad \forall x\,(x = 0 \vee x = S(Px)), \tag{18}$$

$$\mathsf{Sep}: \quad 0 = 1 \longrightarrow \bot, \tag{19}$$

and induction for each *literal equation* $E(n, \vec{p})$:

$$\forall \vec{p}\,\left[(E(0, \vec{p}) \wedge \forall n(E(n, \vec{p}) \longrightarrow E(Sn, \vec{p}))) \longrightarrow \forall n\, E(n, \vec{p})\right]. \tag{20}$$

Here the schematic expression E is a single equality of numerical terms; the displayed universal quantifiers range only over numbers. Every description and every instance of the schema is finite external syntax.

The axiom formulation (20) and the usual contextual induction rule are interchangeable. For example, if

$$\Gamma \vdash E(0, \vec{p}), \qquad \Gamma, E(z, \vec{p}) \vdash E(Sz, \vec{p}),$$

where z is absent from Γ and the passive parameters, discharge the second premise's equation and universally introduce z. The axiom then gives $\Gamma \vdash \forall n E(n, \vec{p})$, which can be instantiated at any target term. Conversely, assume the antecedent of the axiom and apply the contextual rule with fresh variables. This uses the ordinary intuitionistic logical rules and permits arbitrary, including quantified, formulas in Γ.

D.2 Constructor separation and two elementary tests

Define the sign description and the countdown description by

$$\mathrm{sg}(u) = C(u, 0, 1), \qquad K(u) = C(u, 1, Pu). \tag{21}$$

Their defining equations imply

$$\mathrm{sg}(0) = 0, \qquad \mathrm{sg}(Sx) = 1, \tag{22}$$
$$K(0) = 1, \qquad K(Sx) = x. \tag{23}$$

Constructor laws. Successor is injective: applying P to $Sx = Sy$ gives $x = y$ by (16). Successor never equals zero. Indeed, $Sx = 0$ gives $1 = 0$ by applying sg and using (22); symmetry and (19) give $\bot$. Consequently $1 \neq 0$, where $A \neq B$ abbreviates $A = B \to \bot$. None of these arguments uses induction.

The sign test. For every u,

$$\mathrm{sg}(u) = 0 \vee \mathrm{sg}(u) = 1, \tag{24}$$
$$\mathrm{sg}(u) = 0 \quad \longleftrightarrow \quad u = 0, \tag{25}$$
$$\mathrm{sg}(u) = 1 \quad \longleftrightarrow \quad u \neq 0. \tag{26}$$

To prove the range assertion, split (18). The branch $u = 0$ gives sign 0, and the branch $u = S(Pu)$ gives sign 1. The implication $u = 0 \to \mathrm{sg}(u) = 0$ is computation. For its converse, the successor branch of coverage would make $\mathrm{sg}(u) = 0$ imply $1 = 0$, a contradiction; in that branch explosion gives $u = 0$ as well. Similarly, if $\mathrm{sg}(u) = 1$ and $u = 0$, computation gives $0 = 1$. Conversely, $u \neq 0$ excludes the zero branch of coverage, and the successor branch computes the sign as 1.

Coverage and (17) also give, for all u, v,

$$C(u, v, v) = v. \tag{27}$$

Both coverage branches compute to v, so this identity again needs no induction.

The countdown test and the notation W. For every u,

$$K(u) = 0 \quad \longleftrightarrow \quad u = 1. \tag{28}$$

If $u = 1 = S0$, (23) gives $K(u) = 0$. Conversely, assume $K(u) = 0$ and split coverage. If $u = 0$, then $K(u) = 1$, contradicting separation, and explosion gives $u = 1$. If $u = S(Pu)$, computation gives $K(u) = Pu$; hence $Pu = 0$ and $u = S0 = 1$.

If a presentation uses a description named W with defining equations

$$W(0) = 1, \qquad W(Sx) = x,$$

then it proves $K(u) = W(u)$ for every u already from coverage. In the zero branch both sides are 1; in the branch $u = S(Pu)$ both sides are Pu. Thus K and W are provably equal on the whole domain, even if they are distinct descriptions. This equality does not appeal to standard-model agreement or to uniqueness of an arbitrary recurrence.

D.3 Four elementary uses of equation induction

Choose descriptions for addition and truncated subtraction, with recursion on the second argument:

$$x + 0 = x, \qquad x + Sy = S(x + y), \tag{29}$$
$$d(x, 0) = x, \qquad d(x, Sy) = P(d(x, y)). \tag{30}$$

Using these equations and (20), we now derive

$$0 + y = y, \tag{31}$$
$$(Sx) + y = S(x + y), \tag{32}$$
$$d(Sx, Sy) = d(x, y), \tag{33}$$
$$d(Sa, a) = 1. \tag{34}$$

Each induction below has a single equation as its induction formula.

For (31), induct on y. The base is $0 + 0 = 0$. The step is $0 + Sy = S(0 + y) = Sy$.

For (32), hold x fixed and induct on y. At zero both sides are Sx. At a successor the induction hypothesis gives

$$(Sx) + Sy = S\big((Sx) + y\big) = S\big(S(x + y)\big) = S(x + Sy).$$

Combining (32) with the defining equation for addition gives the useful rearrangement

$$(Sy) + r = y + Sr. \tag{35}$$

Indeed, both sides are $S(y + r)$.

For (33), hold x fixed and induct on y. The base computes as

$$d(Sx, S0) = P(d(Sx, 0)) = P(Sx) = x = d(x, 0).$$

Applying P to the induction equation gives the step:

$$d(Sx, S(Sy)) = P(d(Sx, Sy)) = P(d(x, y)) = d(x, Sy).$$

Finally, induct on a for (34). The base is $d(S0, 0) = S0 = 1$. At a successor,

$$d(S(Sa), Sa) = d(Sa, a) = 1,$$

using (33) and the induction hypothesis.

D.4 An equation invariant that reflects actual equality

The remaining arithmetic ingredient is the following equation, valid for all x, y:

$$C(\mathrm{sg}(d(x,y)),\ x,\ y + d(x,y)) = x. \tag{36}$$

Its meaning is that a nonzero subtraction residual reconstructs the original argument by addition. The selector expresses this assertion as one literal equation, so equation induction applies directly.

Hold x fixed and induct on y. At $y = 0$, subtraction gives residual x, and (31) gives $0 + x = x$. The desired equation is consequently $C(\mathrm{sg}(x), x, x) = x$, which is (27).

For the step, assume (36) at y and abbreviate

$$r = d(x,y), \qquad r' = Pr = d(x, Sy).$$

These are local term abbreviations, not additional function symbols. Split coverage for r'.

If $r' = 0$, then $\mathrm{sg}(r') = 0$, and the successor instance of (36) reduces immediately to $x = x$.

In the other branch, $r' = S(Pr')$, so constructor separation gives $r' \neq 0$. Split coverage for r. Its zero branch would imply $r' = P0 = 0$, a contradiction, from which the desired step equation follows. In its successor branch,

$$r = S(Pr) = Sr', \qquad \mathrm{sg}(r) = 1.$$

The induction equation therefore computes to $y + r = x$. Meanwhile, (35) gives

$$(Sy) + r' = y + Sr' = y + r = x.$$

Since r' is a successor, $\mathrm{sg}(r') = 1$, so this is exactly the branch selected in the successor instance of (36). This completes the equation induction. All case distinctions came from coverage; no induction on an implication or a disjunction was used.

Reflection at residual 1. We have, for arbitrary a, b,

$$d(Sa, b) = 1 \quad \longleftrightarrow \quad a = b. \tag{37}$$

For the forward direction, substitute $x = Sa$, $y = b$ into (36). If $d(Sa, b) = 1$, the sign is 1 and the selector returns its second branch, giving

$$b + 1 = Sa.$$

The defining equation for addition gives $b + 1 = S(b + 0) = Sb$. Thus $Sb = Sa$, and applying P gives $b = a$, hence $a = b$. Conversely, $a = b$ and (34) give $d(Sa, b) = d(Sa, a) = 1$.

Define the equality characteristic by the fixed description

$$e(a,b) = \mathrm{sg}\big(K(d(Sa,b))\big). \tag{38}$$

The sign, countdown, and residual reflection lemmas give

$$e(a,b) = 0 \quad \longleftrightarrow \quad a = b, \tag{39}$$

$$e(a,b) = 0 \ \vee\ e(a,b) = 1. \tag{40}$$

In detail, $e(a,b) = 0$ is equivalent to $K(d(Sa,b)) = 0$ by (25), which is equivalent to $d(Sa,b) = 1$ by (28), and then to $a = b$ by (37). The range assertion is (24) applied to $K(d(Sa,b))$.

The reflection property concerns the logic's actual equality throughout the domain. It was not assumed as part of a numerical comparison symbol, and its proof did not restrict a or b to standard numerals. It also yields decidable equality. In the $e(a,b) = 0$ branch, (39) gives $a = b$. In the $e(a,b) = 1$ branch, an assumption $a = b$ would give $e(a,b) = 0$ and therefore $0 = 1$, contradicting separation. Hence

$$\forall a\, \forall b\, \big(a = b \vee (a = b \to \bot)\big) \tag{41}$$

is a theorem, not an extra hypothesis.

D.5 Compilation of all quantifier-free formulas

For each fixed quantifier-free formula $A(\vec{p})$, construct a term $b_A(\vec{p})$ by external recursion on its finite syntax. Take equality, $\bot$, $\wedge$, $\vee$, and $\to$ as the primitive formula constructors; negation is implication into $\bot$. Set

$$b_{s=t} = e(s,t), \qquad b_\bot = 1, \tag{42}$$
$$b_{A\wedge D} = C(b_A, b_D, 1), \tag{43}$$
$$b_{A\vee D} = C(b_A, 0, b_D), \tag{44}$$
$$b_{A\to D} = C(b_A, b_D, 0). \tag{45}$$

Every term here is a finite composition of fixed primitive-recursive descriptions; if desired, projections and composition give it a single description symbol. No internal quantification over formulas or programs is involved.

An external induction on the syntax of A proves, within the arithmetic theory,

$$\big(b_A = 0 \vee b_A = 1\big) \quad \text{and} \quad \big(b_A = 0 \longleftrightarrow A\big). \tag{46}$$

Here are the full connective checks. In every case $b_A = 1$ implies $\neg A$: if A held, the already established implication $A \to b_A = 0$ would contradict separation.

Atomic formulas and falsity. For an equation, both claims are (39)–(40). For falsity the range assertion is $1 = 0 \vee 1 = 1$, with its right disjunct proved by reflexivity. Also $1 = 0 \to \bot$ follows from separation and symmetry, while $\bot \to 1 = 0$ follows by explosion. Thus $1 = 0$ is equivalent to $\bot$, as required.

Conjunction. Split $b_A = 0 \vee b_A = 1$. In the zero branch, A holds and $b_{A\wedge D} = b_D$. Its range follows from the range of b_D, and its being zero is equivalent to D, hence to $A \wedge D$ under the known A. In the one branch, $\neg A$ holds and $b_{A\wedge D} = 1$. Its range is immediate. If it were zero, separation would give $\bot$, hence $A \wedge D$; conversely an assumption $A \wedge D$ contradicts $\neg A$, and explosion gives $b_{A\wedge D} = 0$.

Disjunction. In the branch $b_A = 0$, the formula A holds and $b_{A\vee D} = 0$. The range is immediate, and both $b_{A\vee D} = 0$ and $A \vee D$ hold, proving their equivalence. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\vee D} = b_D$. Its range follows from the range of b_D. If its value is zero, then D and hence $A \vee D$. Conversely, from $A \vee D$, the A alternative contradicts $\neg A$ and yields $b_D = 0$ by explosion, while the D alternative yields $b_D = 0$ directly. This proves the required equivalence in both alternatives.

Implication. In the branch $b_A = 0$, the formula A holds and $b_{A\to D} = b_D$. Its range follows from the range of b_D. If its value is zero, then D, which gives $A \to D$. Conversely, $A \to D$ and the known A give D, hence $b_D = 0$. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\to D} = 0$. Its range is immediate. To prove $A \to D$, assume A; this contradicts $\neg A$, and explosion gives D. Thus both $b_{A\to D} = 0$ and $A \to D$ hold in this branch, and their equivalence follows. This argument uses the proved two-value range of the finite characteristic. It does not use excluded middle for arbitrary formulas or change the surrounding intuitionistic logic.

This completes the external syntactic induction proving (46) for every quantifier-free formula.

D.6 Recovery of induction and the full theory

Let $A(n, \vec{p})$ be any fixed quantifier-free formula. Suppose an arbitrary retained context Γ proves its base and its induction step:

$$\Gamma \vdash A(0, \vec{p}), \qquad \Gamma, A(z, \vec{p}) \vdash A(Sz, \vec{p}),$$

with the usual freshness condition on z. By (46), the base gives $\Gamma \vdash b_A(0, \vec{p}) = 0$. Under the additional assumption $b_A(z, \vec{p}) = 0$, the same equivalence gives $A(z, \vec{p})$, the given step gives $A(Sz, \vec{p})$, and the equivalence gives $b_A(Sz, \vec{p}) = 0$. Apply literal-equation induction to $b_A(n, \vec{p}) = 0$ and then apply its reflection implication. The result is $\Gamma \vdash A(t, \vec{p})$ for every target term t. Thus the theory proves the full contextual quantifier-free induction rule, including in arbitrary quantified contexts.

Completion theorem. In the common language of fixed primitive-recursive descriptions, the theory

$$\mathsf{PRE} + \mathsf{Cov} + \mathsf{Sep} + \mathsf{EI}$$

has exactly the same sequents as ordinary first-order intuitionistic primitive recursive arithmetic with all quantifier-free induction instances. The forward simulation follows from the recovered quantifier-free induction, successor injectivity, and successor separation; all primitive-recursive defining equations and all logical rules are shared. For the reverse simulation, literal-equation induction is an instance of quantifier-free induction, and successor separation gives Sep. Quantifier-free induction proves Cov: at zero choose its left disjunct, and at a successor use $Sx = S(P(Sx))$, which follows from $P(Sx) = x$, for its right disjunct.

Both interpretations therefore use the full domain, defined by $x = x$, and preserve every function symbol, equality, logical connective, and quantifier. Both composites have the identity isomorphism, with graph $x = y$. The theories are consequently identity-domain bi-interpretable and equiconsistent. This completion theorem is unconditional once the literal-equation induction rule has been obtained; it has no dependency on a classical successor-uniqueness reduction.

E Pairing, primitive recursion, and internal numerical composites

This appendix is adapted with attribution from the progress-closure arithmetic development [12]. Its prerequisites have now been proved: scalar predecessor, arithmetic reflection, and full quantifier-free induction. Pairing is a derived proof tool; it is not part of any checker.

E.1 Pairing from a bounded square-root search

Once QFI is available, a familiar square-based injection gives a particularly direct compiler. Write $a \le b$ for $d(a, b) = 0$. The usual linear-order and semiring identities follow by fixed quantifier-free

inductions from the successor, addition, and monus equations. Multiplication itself is the scalar term

$$a \cdot b = I(b, 0; u.u + a).$$

Only primitive-recursive arithmetic is involved in the following bounded root search; there is no minimization over an unbounded domain.

For a fixed parameter n, put

$$U_n(v) = C(d((Sv)^2, n), Sv, v), \qquad R_n(i) = I(i, 0; v.U_n(v)), \qquad q(n) = R_n(n).$$

The update increments v exactly when $(v+1)^2 \leq n$, and otherwise retains it. QFI in i, with n fixed, proves

$$R_n(i) \leq i, \qquad R_n(i)^2 \leq n, \qquad R_n(i) = i \ \vee \ n < (S(R_n(i)))^2. \tag{Root invariant}$$

The base has $R_n(0) = 0$. If the update increments, its test contradicts the strict-upper-bound alternative in the third invariant. Thus the old state equals i, and the new state equals Si; its square bound is the increment test. If the update does not increment, the strict upper square bound holds and the state remains fixed. An already established strict upper square bound forces every subsequent update to remain fixed. These are exactly the cases needed for the three displayed invariants.

At $i = n$, either the strict upper square bound already holds or $R_n(n) = n$, in which case $n < (n+1)^2$ supplies it. Thus

$$q(n)^2 \leq n < (q(n)+1)^2. \tag{Root}$$

Monotonicity of squares shows uniqueness of a number satisfying Root. All formulas just used are quantifier free in the scalar language, with fixed n during the induction.

Now define

$$\pi(a, b) = (a + b)^2 + a, \qquad L(n) = d(n, q(n)^2), \qquad H(n) = d(q(n), L(n)).$$

For $k = a + b$, one has $k^2 \leq k^2 + a < (k+1)^2$ since $a \leq k$. Root uniqueness gives $q(\pi(a, b)) = a + b$, and therefore

$$L(\pi(a, b)) = a, \qquad H(\pi(a, b)) = b.$$

Surjectivity of π is unnecessary. The original numerical domain is never replaced by the range of this coding; pairing is used only to store the finite state of a fixed primitive recursion. This standard square pairing is a convenient reconstruction here, not a claimed new arithmetic basis.

E.2 Compiling each primitive recursion

Suppose the fixed target description is

$$f(0, \vec{p}) = g(\vec{p}), \qquad f(Sn, \vec{p}) = h(n, f(n, \vec{p}), \vec{p}).$$

Assume by external structural induction that the earlier g, h already have scalar translations g^*, h^*. Define

$$W(u, \vec{p}) = \pi(SL(u), h^*(L(u), H(u), \vec{p})),$$

$$V(n, \vec{p}) = I(n, \pi(0, g^*(\vec{p})); u.W(u, \vec{p})), \qquad f^*(n, \vec{p}) = H(V(n, \vec{p})).$$

The update has only its state variable bound, with every p_i fixed. Equation induction proves $L(V(n, \vec{p})) = n$. The two projection equations and scalar computation then prove both defining equations for f^* internally. Initial functions and composition translate directly. This completes the translation of every fixed finite PR description without adding a universal evaluator.

E.3 Why both numerical round trips are internal

Two fixed scalar iterators with the same seed and provably equal updates are internally equal at all counters: use literal-equation induction comparing their values. Their base agrees, and their successor equations plus congruence prove the step. The same uniqueness argument works for ordinary primitive recursion.

For an ordinary PR description, induct externally on its finite construction. The double translation satisfies its original defining equations, so internal equation induction identifies it with the original operation. For a native scalar description, earlier translated updates agree by the external induction hypothesis; scalar uniqueness identifies the double-translated iterator. These proofs hold at arbitrary internal counters.

Thus the domain relation $x = y$ is provably a total bijection preserving every fixed numerical operation in both composites. The formula constructions and round trips in Section A give the corresponding formula composites. Both interpretations preserve falsity, taking native $0 = 1$ to an ordinary contradiction and ordinary $\bot$ to native $0 = 1$. Finite proof translations therefore establish equiconsistency.

References

[1] ProveIt contributors. ConciseIPRA, pinned repository snapshot `b17dd5487f34adea473a0204c5320fb3eeb8910e`. https://github.com/VladimirReshetnikov/ProveIt/tree/b17dd5487f34adea473a0204c5320fb3eeb8910e/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA.

[2] ProveIt, *neutral_scope*, complete byte-profile checker and report, same pinned snapshot, `Research/neutral_scope/`.

[3] ProveIt, *prevalidated_elimination*, `Research/prevalidated_elimination/code/checker_structure.py`, same pinned snapshot.

[4] ProveIt, *coverage_and_kernel*, `Research/coverage_and_kernel/code/checker_ast.py`, same snapshot.

[5] ProveIt contributors. `Lean/ConciseIPRA/BiInterpretation.lean`, same pinned snapshot; whole-domain scalar/iPRA interpretations and consistency equivalence.

[6] ProveIt contributors. `Lean/ConciseIPRA/NativeComposite.lean`, same pinned snapshot; native round trip and `NPrf.iff_teq`.

[7] ProveIt research series, equality-reflection argument attributed by the inherited text to the forward-left draft; reproduced here from the pinned file `Research/certified_zero/reference/equality_reflection.tex`.

[8] ProveIt research series, generated-assumption and Choice-DI basis, preserved in `Research/certified_zero/reference/inherited_choice_basis.tex`.

[9] ProveIt research series, `Research/constructed_computation/`, same pinned snapshot.

[10] ProveIt research series, the curated `Logic/PrimitiveRecursiveArithmetic/ConciseIPRA/Research/` corpus, same pinned snapshot.

[11] ProveIt research series, scalar predecessor bootstrap attributed by the inherited source to the unparenthesized one-hole development; reproduced here from `Research/certified_zero/reference/scalar_predecessor.tex` at the pinned snapshot.

[12] ProveIt research series, progress-closure arithmetic development; the inherited pairing and recursion argument is reproduced here from `Research/certified_zero/reference/primitive_recursion.tex` at the pinned snapshot.